

就活作品



就活作品

作品名 : Rhythm Strike!!

ジャンル : 3Dリズムアクション

開発環境 : C#,Unity

制作期間 : 2025年9月～2026年2月

制作人数 : 1人

GitURL : <https://github.com/Ayu20060301/Rhythm-Strike>

動画URL : <https://youtu.be/3ogXOeafxxk>



作品概要

ジャンル

リズム



アクション

Rhythm
Stroke!!



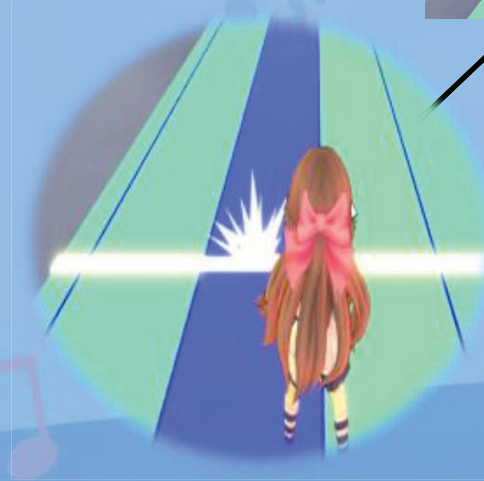
リズムに乗って
敵や障害物を越えよう!

このゲームの面白さ①

- ・ リズムに合わせた攻撃アクション
- ・ ノーツをタイミングよく押して敵を攻撃
- ・ コンボをつなぐ爽快感

攻撃

Rhythm
Strike!!



このゲームの面白さ②



- ・ 障害物を避けるアクション
- ・ ジャンプで障害物を回避



←ジャンプ

避ける→



技術紹介

```

    Note in inputJson.notes;

    NoteData = DataBaseManager.Instance.NoteData;
    if (NoteData == null)

    Debug.LogWarning($"NoteData is null, continue;");

    // 番号 / LPB * 小節の長さ
    float time = (float)note.block * (60.0f / inputJson.BPM) + inputJson.offset * 0.001f;

    // 追加
    _notes.Add(note.block);
    _types.Add(note.type);
    _times.Add(time);

    // DZ座標(時間 * 小節の長さ)
    Vector3 position = new Vector3(time * _notesSpeed, 0, 0);

    Instantiate(
        _notePrefab,
        position,
        Quaternion.identity,
        _notePrefab.transform.rotation);
    
```

```

    // データベースを一括管理するクラス
    public class DataBaseManager : SingletonMonoBehaviour<DataBaseManager>
    {
        // データベースを入れる”
        private static DataBaseManager _instance;

        public static DataBaseManager Instance { get; }

        private DataBaseManager()
        {
            _noteDataBase = new NoteDataBase();
            _soundDataBase = new SoundDataBase();
            _uiDataBase = new UiDataBase();
            _songDataBase = new SongDataBase();
            _particleDataBase = new ParticleDataBase();
            _movieDataBase = new MovieDataBase();
        }

        // プロパティ読み込み可能
        public NoteDataBase noteDB => _noteDataBase;
        public SoundDataBase soundDB => _soundDataBase;
        public UiDataBase uiDB => _uiDataBase;
        public SongDataBase songDB => _songDataBase;
        public ParticleDataBase particleDB => _particleDataBase;
        public MovieDataBase movieDB => _movieDataBase;

        public void Load()
        {
            // ...
        }
    }
    
```

技術紹介① : 譜面作成 ※使用したNoteEditor

➡ <https://github.com/setchi/NoteEditor>



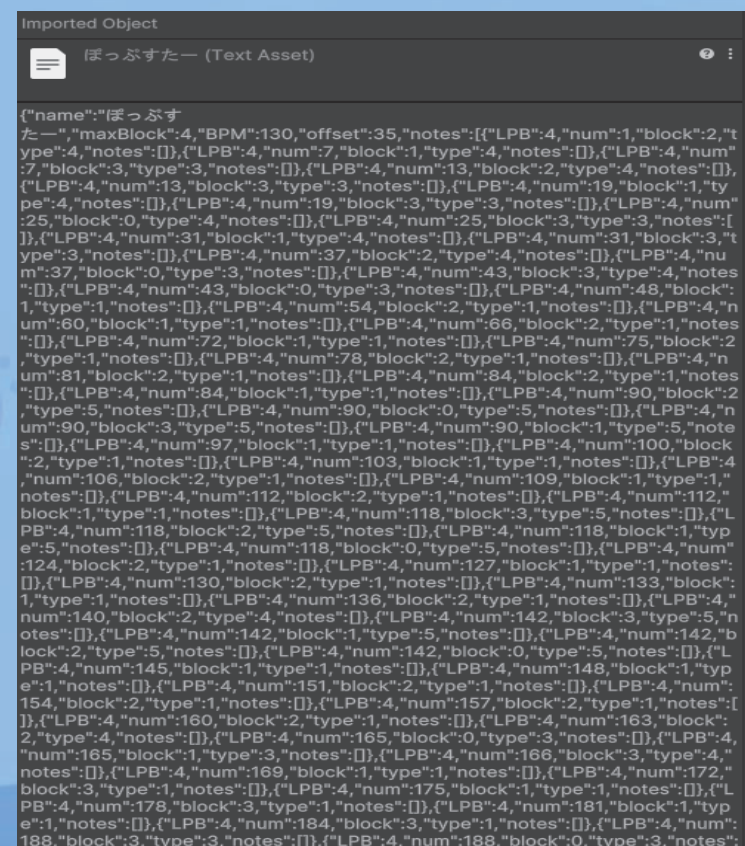
NoteEditorという外部Editorを使用して
譜面を作成しました。

下の画像のEditorで譜面の情報を入れ
Jsonファイルで保存されます。

▼実際に使用しているEditor



▼読み込んだJsonファイルの中身



技術紹介① : NoteEditorの拡張

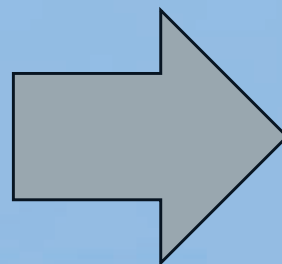
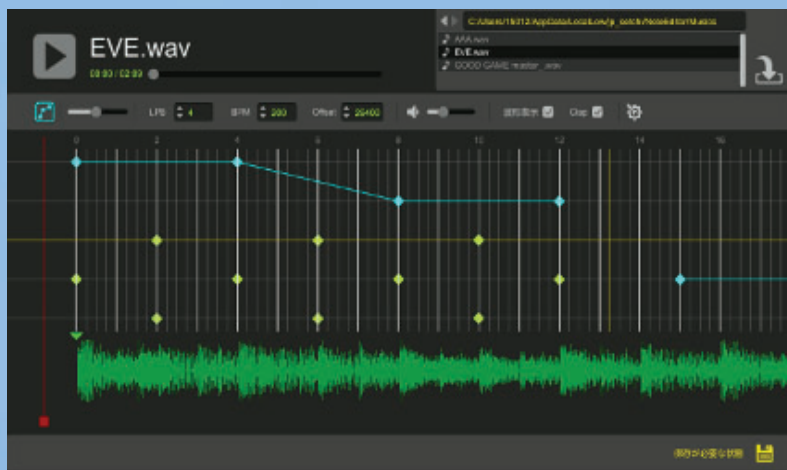


既存のNoteEditorでは配置できるノーツの種類が限定されていたため、**Editorの改良**を行い、新たなノーツタイプを追加できる仕組みを実装しました。

これにより**譜面制作の自由度**を向上させました。

▼元のNoteEditor

☆ノーツの種類が2種類しかなかった



▼改良後のNoteEditor

★ノーツの種類が5種類に増えた



技術紹介② : ノーツの生成 (1 / 2)



Jsonファイルから曲ごとのノーツ情報を読み込み、判定線に到達する時間を計算してPrefabを生成するクラスを作成しました。

ノーツの種類・レーン・判定時間を管理することで、スコアやコンボ計算に必要な情報を提供しています。

▼NotesManager.cs参照

```
float secPerBeat = 60.0f / inputJson.BPM; //1拍
float secPerBar = secPerBeat * 4.0f; //1小節(4拍)

foreach(var note in inputJson.notes)
{
    var noteData = DataBaseManager.instance.noteDataBase.GetNoteByData(note.type);
    if(noteData == null)
    {
        Debug.LogWarning($"NoteDataが見つかりません type={note.type}");
        continue;
    }

    //time = (番号 / LPB) * 小節の長さ + offset
    float time =
        note.num * (60.0f / inputJson.BPM / note.LPB)
        + inputJson.offset * 0.001f;

    //リストに追加
    laneNum.Add(note.block);
    noteType.Add(note.type);
    notesTime.Add(time);

    //ノーツのZ座標(時間 × 速度)
    float z = time * _notesSpeed;

    var obj = Instantiate(
        noteData.prefab,
        new Vector3(note.block - 1.5f, noteData.y, z),
        noteData.prefab.transform.rotation
    );

    notesObj.Add(obj);
}
```

技術紹介② : ノーツの生成 (2 / 2)



Jsonデータに基づき、各ノーツが正しいレーン・タイミングで生成されます。

ノーツの種類ごとにPrefabを使い分け、判定線に到達するタイミングまで正確に落下します。

▼生成されたノーツ



技術紹介③ : ノーツの基底クラス(NotesBase)

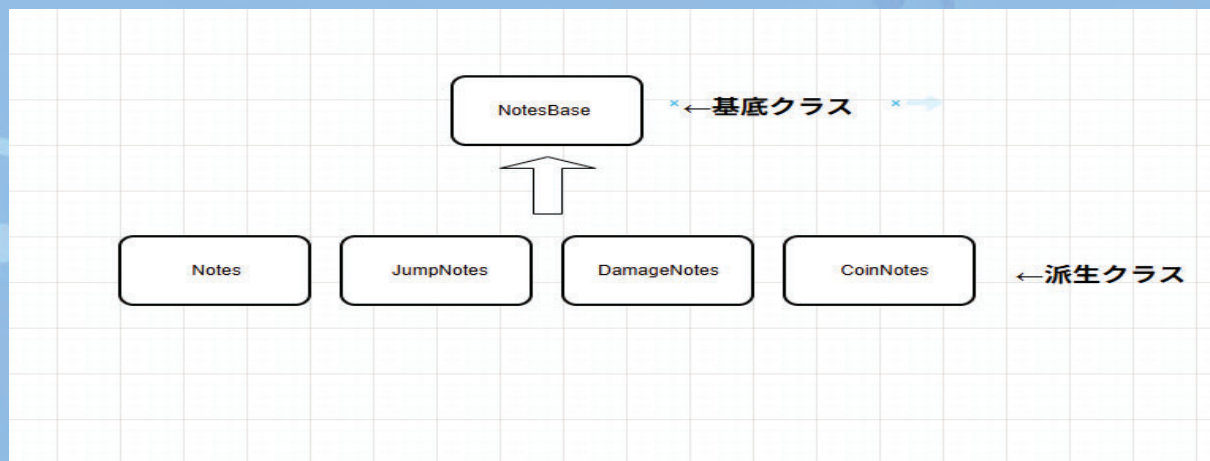


ノーツ共通の挙動をまとめた基底クラスを実装しました。

ノーツごとのクラス分けを行ったことにより、可読性と管理のしやすさが向上しています。

また、今後のノーツ追加や変更にも柔軟に対応できる拡張性の高い設計になっています。

▼簡易なクラス図



技術紹介④ : ノーツ判定システム



プレイヤー入力に応じたノーツ判定を管理するクラスを実装しました。

Critical、Hit、Attackの判定をms単位で制御するようにしています。

▼判定の許容時間

```
/// <summary>
/// ノーツ判定を管理するクラス
/// </summary>
/// </summary>
/// Unity スクリプト (1 件のアセット参照) 10 個の参照
public class Judge : MonoBehaviour
{
    //判定の許容時間
    private const float _kCriticalTime = 0.025f; //25ms
    private const float _kHitTime = 0.05f; //50ms
    private const float _kAttackTime = 0.08f; //80ms
}
```

▼Judge.cs参照

```
Assembly-CSharp
Judge
195 // <summary>
196 // ノーツの判定処理
197 // </summary>
198 // <param name="timeLag">たいた時間の誤差</param>
199 // <param name="noteIndex">対象ノーツのインデックス</param>
200 // 1 個の参照
201 void Judgement(float timeLag, int noteIndex)
202 {
203     //判定範囲外の場合は処理をしない
204     if (timeLag > _kAttackTime) return;
205
206     //インデックスの範囲チェック
207     if (!IsValidNoteIndex(noteIndex)) return;
208
209     //シングルノーツ以外は処理をしない
210     int noteType = _notesManager.noteType[noteIndex];
211     if (noteType != _kNoteTypeSingle) return;
212
213     JudgeType judgeType;
214
215     //判定の決定
216     if (timeLag <= _kCriticalTime + GManager.instance.timingOffset * 0.01f)
217     {
218         //Critical
219         judgeType = JudgeType.CRITICAL;
220     }
221     else if (timeLag <= _kHitTime + GManager.instance.timingOffset * 0.01f)
222     {
223         //Hit
224         judgeType = JudgeType.HIT;
225     }
226     else if (timeLag <= _kAttackTime + GManager.instance.timingOffset * 0.01f)
227     {
228         //Attack
229         judgeType = JudgeType.ATTACK;
230     }
231     else
232     {
233         return;
234     }
235     HandleHit(judgeType, noteIndex);
236 }
```

技術紹介⑤ : マネージャー(GManager)



ゲーム全体の進行やスコア、判定、設定などを一元管理するクラスを実装しました。

Singletonパターンを用いることで、各クラスから安全にアクセスできる構成にしています。
また、ゲーム状態(開始・ポーズ・終了・死亡)を明確に管理し、安定した進行制御を実現しました。

▼GManager.cs参照

```
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
// <summary>
// ゲーム全体の進行を管理するクラス
// SingletonMonoBehaviourを継承
// どのクラスにもアクセス可能にする
// </summary>
// Unity スクリプト (1 件のアセット参照) 199+ 個の参照
public class GManager : SingletonMonoBehaviour<GManager>
{
    [Header("進行管理")]
    public int songID; //曲のID
    public float notesSpeed; //ノーツスピード
    public float timingOffset; //タイミングオフセット
    public bool isStart; //ゲームが開始したかどうかのフラグ
    public float startTime; //ゲーム開始時間
    public bool isDeath; //死亡したかどうかのフラグ
    public bool isEnd; //ゲームが終了したかどうかのフラグ

    [Header("スコア関連")]
    public int score;
    public float maxScore;
    public float ratioScore; //スコアの比率

    [Header("コンボ数関連")]
    public int combo;
    public int maxCombo;

    public bool isComboBrokenOnce; //コンボが途切れたかどうかのフラグ

    [Header("判定")]
    public int critical;
    public int hit;
    public int attack;
    public int miss;
    public int coin;
    public int maxCoin;

    [Header("音量")]
    public float masterVolume;
    public float bgmVolume;
    public float seVolume;
    public float musicVolume;
```


技術紹介⑥ : データベース(1 / 2)



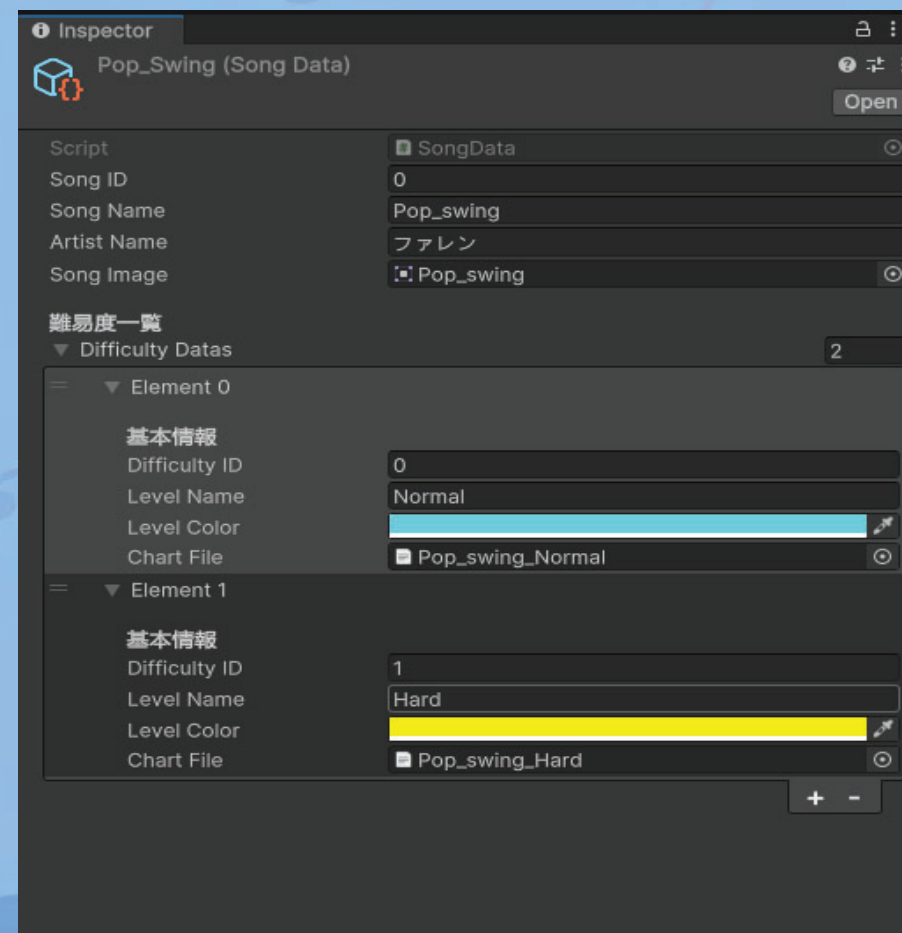
楽曲ごとの情報(譜面・画像、曲名など)を
ScriptableObjectとして管理し、
新規楽曲情報追加時にコード修正なしでデータ
追加のみで対応できる設計にしました。

これにより、制作効率と拡張性を大きく向上
させました。

▼実際のゲーム画面



▼曲のデータを入れる項目



技術紹介⑥ : データベース(2 / 2)

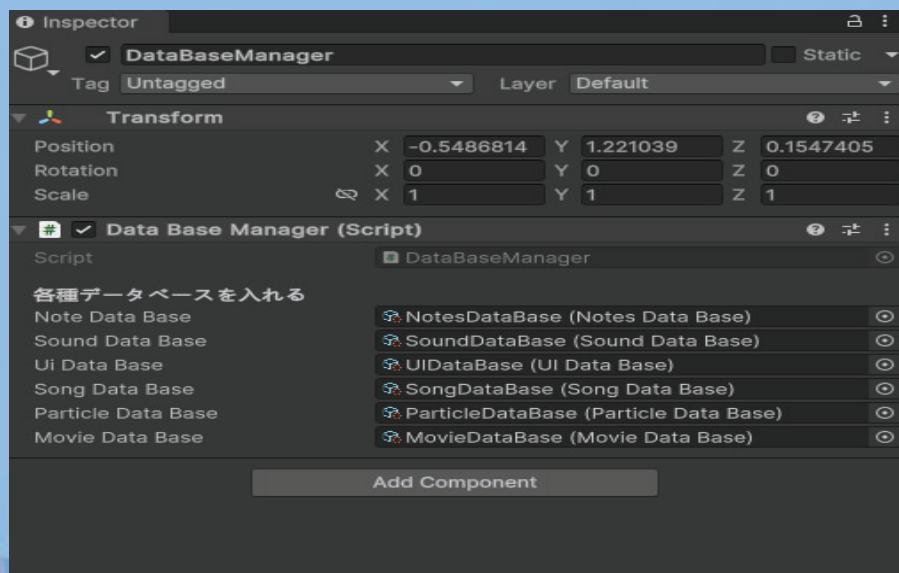


各データベースを一元管理するクラス

DataBaseManagerを実装しました。

Singletonパターンを用いることで、各クラスから個別にデータベースを参照する必要がなく、**インスペクター上でデータを設定する**だけで利用する設計にしています。

▼各種データベースを入れる項目



▼DataBaseManager.cs参照

```
using System;
using System.Collections;
using System.Collections.Generic;
using Unity.VisualScripting;
using UnityEngine;

/// <summary>
/// ゲーム全体で使うデータベースを一括管理するクラス
/// </summary>
/// Unity スクリプト (1 件のアセット参照) 116 個の参照
public class DataBaseManager : SingletonMonoBehaviour<DataBaseManager>
{
    [Header("各種データベースを入れる")]
    public NotesDataBase noteDataBase; //ノーツ関連のデータ
    public SoundDataBase soundDataBase; //サウンド関連のデータ
    public UIDataBase uiDataBase; //UI関連のデータ
    public SongDataBase songDataBase; //曲関連のデータ
    public ParticleDataBase particleDataBase; //パーティクル関連のデータ
    public MovieDataBase movieDataBase; //Movie関連のデータ

    //外部からのプロパティ読み込み可能
    0 個の参照
    public NotesDataBase noteDB => noteDataBase;
    2 個の参照
    public SoundDataBase soundDB => soundDataBase;
    5 個の参照
    public UIDataBase uiDB => uiDataBase;
    4 個の参照
    public SongDataBase songDB => songDataBase;
    1 個の参照
    public ParticleDataBase particleDB => particleDataBase;
    0 個の参照
    public MovieDataBase movieDataDB => movieDataBase;

    10 個の参照
    protected override void Awake()
    {
        base.Awake();
    }
}
```

技術紹介⑦ : シェーダー(1 / 2)



ロード画面の演出として、ストライプ模様が任意の角度で流れるアニメーションをシェーダーで再現しました。

滑らかな演出を再現し、プレイヤーの待機時間のストレス軽減を意識しています。

▼実際のロード画面
※ゲーム画面では流れてます!



技術紹介⑦ : シェーダー(2 / 2)



← ロード画面の説明の続き...

UV座標をベクトル方向に射影し、
時間によるスクロールを加えることで
軽度なアニメーションを実現しました。

▼BG_Load.shader参照

```
61         return o;
62     }
63
64     fixed4 frag (v2f i) : SV_Target
65     {
66         //角度 → ラジアン
67         float rad = radians(_Angle);
68
69         //方向ベクトル
70         float2 dir = float2(cos(rad),sin(rad));
71
72         //斜め方向の座標
73         float stripeCoord =
74             dot(i.uv,dir) * _Density
75             + _Time.y * _Speed;
76
77         //周期化
78         float stripe = frac(stripeCoord);
79
80         //アンチエイリアス
81         float aa = fwidth(stripe);
82
83         //ストライプマスク
84         float mask = smoothstep(
85             _Width - aa,
86             _Width + aa,
87             stripe
88         );
89
90         return lerp(_ColorA,_ColorB,mask);
91     }
92
93     ENDRC
```

作品総括

まだまだ **遊び要素が少ない!!**

上記の問題を解決するためには

- ・ 曲を増やす
- ・ 操作性の改善
- ・ ギミックに合わせた演出の強化